

CHAPTER 3: POPULATION-BASED MODELING

3.1 Introduction

ABM models can represent discrete cells and stochastic behaviors, but they are prohibitively resource intensive on a large scale. As can be seen in the performance scaling tests in the Appendix Section 8.1.2, ABM run time scales approximately linearly with the number of agents. This is a fine solution when the number of cells is below 1 million or so. However as the number of cells scales up, eventually memory limitations become an issue. At 100 million or more cells, the laptop that ran the tests was unable to execute the model. Regardless of the eventual breaking point, this direct approach of simulating every cell becomes problematic at scale, and the scale of tumors is immense from a cell perspective. Assuming around 1 billion cells per cm^3 [107] (a reasonably detectable size of a tumor on imaging), just one cm^3 of cells is already well past what is feasible on most hardware [20]. In order to continue modeling with discrete cells and stochastic mechanistic rules at this scale, a different approach is necessary.

When dealing with large numbers of cells it is often useful to model the spatial-temporal cellular dynamics as densities using PDEs [39, 40, 108, 109]. This approach is effective in modeling tumor dynamics, such as turnover, migration, and changing phenotype. However, it can only model these phenomena using densities, and does not directly represent individuals nor rare stochastic events. To bridge between PDE scalability and ABM stochasticity, we introduce the PopulationGrid.

The PopulationGrid is similar to the AgentGrid discussed in the previous chapter, but it stores homogeneous populations at each lattice position rather than storing individual agents. The binomial and multinomial functions are used to update the populations in response to probabilistic events [91]. These events may consist of migration, proliferation, death, or changes in agent state (eg. mutation, phenotype changes, etc.). The benefit of this type of grid is that it can efficiently handle arbitrarily large populations at each lattice position (up to the maximum size of a Java Long $\approx 9 \times 10^{18}$). The downside is that all of the agents at one lattice position in a given

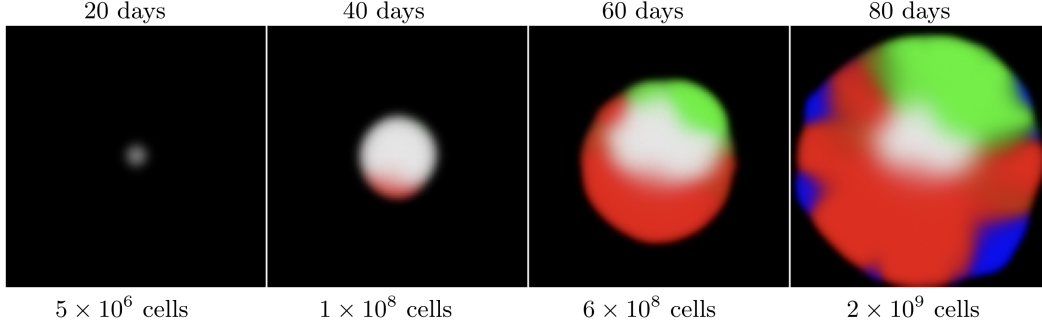


Figure 8. An example PopulationGrid model of tumor growth and mutation in 2D (located in Examples/PopulationTumor within the HAL source code). At the start 1000 cells are placed in the center of the domain. Cells turn over and migrate over time. Two proliferation-increasing mutations are possible, and can occur with probability 2×10^{-6} per cell division. The colors white, red, green, and blue indicate density of cells with no mutations, one mutation, the other mutation, and both mutations respectively. Each lattice position in the domain has a carrying capacity of 100,000 cells. If we assume that each pixel represents an area of 1 mm^2 , and that the carrying capacity of 1 mm^3 is a million cells, then the total domain size is $150 \times 150 \times 0.1 \text{ mm}$. Displayed using the GridWindow object.

PopulationGrid are identical. The Population-based modeling approach was independently developed and named mesoscopic modeling by the Perez-Garcia lab [27]. Their fundamental approach is the same as ours.

Although all agents at a given location within a PopulationGrid are identical, changes in agent state can be implemented by shifting agents between multiple PopulationGrids. Figure 8 demonstrates how this approach may be used to model stochastic occurrences of very unlikely events over large populations, such as specific driver mutations occurring in a growing tumor. This example model uses 4 PopulationGrids, one grid for each mutation state that a cell can take: no mutations (grey), one growth accelerating mutation (red), another growth accelerating mutation (green), both mutations (blue). The cells start in the center of the domain with no mutations. When a cell acquires one of the growth mutations, it switches to a different PopulationGrid. Despite the cells existing on 4 separate grids, they all share a carrying capacity at each location that limits proliferation, thus causing competition for space.

3.2 PopulationGrid implementation

The PopulationGrid object can be used for a variety of modeling approaches, much like the rest of HAL’s components. However, in this thesis, the PopulationGrids are used within a specific

conceptual framework. We will first discuss the implementation of the PopulationGrids object and its generic capabilities, and then discuss how these capabilities are used to build the models in the next two chapters.

Fundamentally, the PopulationGrid consists of an array of integer values which store the agent count or "population" at each grid point. The primary operation on these populations is done by binomial/multinomial distribution sampling to determine the numbers of agents that will take certain probabilistic state changes. A tested calculator for this is included as a tool in HAL. This binomial/multinomial calculator returns a number of agents that will change, which results in changes to the array of populations. This is the core of the PBM paradigm.

These changes to the PopulationGrid do not happen immediately, however. In order to ensure that the order of evaluation of the voxels and probabilistic events does not bias the behavior of the model, the PopulationGrid is designed to update changes over all voxels in at once. The PopulationGrid contains an integer array of the same dimension as the array of populations that stores the "deltas" or changes to each grid point before they are applied. When all of the desired changes have been added to the deltas array, an Update function is called that applies the changes to the PopulationGrid population array. In this way, the PopulationGrid object behaves similarly to the HAL PDE solver, where changes are calculated using the current state of the model, then the model is updated with these changes.

To solidify these concepts, we will discuss an example of using the binomial solver to calculate the number of cells that will divide at a given voxel. The voxel starts with positive integer population n and probability of division p per cell. The binomial solver takes n and p as inputs and returns an integer $a \leq n$ of agents that should be added to the population. The modeler then takes this number of agents and alters the deltas array of the PopulationGrid according to this change, adding a to the deltas for that voxel. When the Update function is called on the PopulationGrid, the change in population will be applied.

We will next discuss the case of using the multinomial solver to calculate the number of agents in a starting voxel that will move to several adjacent voxels. In this case the population must be divided between several mutually exclusive probabilistic options. Suppose that all of the probabilities of moving into each adjacent voxel are contained in a vector $\bar{p}$ whose sum is ≤ 1 . The solver is first initialized with the total population size n at the starting voxel. Next, one by

one from $\bar{p}$, we take a probability entry p_i , and plug it into the multinomial solver to get the sub-population a_i that will move to one of the adjacent voxels. For each a_i the modeler moves this number of agents from the starting voxel to the adjacent voxel (which internally is just a subtraction from the deltas array at the starting voxel and an addition to the deltas array at the adjacent voxel). The multinomial solver internally keeps track of the remaining population from n that has not yet been allocated as well as the sum of all probabilities that have been entered, such that the modeler just has to enter the next p_i to get the next a_i until all movements to adjacent voxels have been calculated.

3.3 PopulationGrid performance

There were several optimizations added to the PopulationGrid to increase its performance in different situations. The binomial solver that HAL uses is a combination of an implementation found in the Colt java mathematics library [100], and the scipy stats [110] binomial distribution sampler. We found that the Colt solver was faster, but for small population sizes and extreme probabilities $np < 30$ or $n * (1 - p) < 30$ it was not producing accurate distributions. To fix this we substituted the scipy stats version in these cases.

An important performance gain was made by comparing the time it took to run the binomial solver to the time it took to sample the probability n independent times using HAL's random number generator, analogous to "sequential coin flips". We found that the performance of the random number generator was faster until $n > 10$. So if $n \leq 10$ we use the random number generator, otherwise we use the binomial solver. This is why the performance of the PBM in Figure 9 is faster when the number of agents per lattice position is 1 and 10, but flattens out at 100 and 1000 agents.

We also looked at the performance that could be gained in very sparse PopulationGrids, where there are only a few voxels with nonzero populations. In that case, it may be faster to store where the populated voxels are in a list and iterate over just those locations specifically, rather than iterating over every location. We found that the location list version was faster when the number of grid points that needed to be visited was $< 1\%$ of the total number of grid points. With this information, we setup the PopulationGrid to work as follows: By default, the PopulationGrid will iterate over every location. If the number of occupied locations is $< 0.1\%$ of the total number of

grid points, the PopulationGrid will switch to using a populated location list. If the number of occupied locations $> 1\%$, the PopulationGrid will switch back to iterating over every location. Switching to a location list at 0.1% instead of 1% prevents frequent switching between full iteration and a populated location list, which has a performance cost of its own, while ensuring that extra effort is not wasted on iterating over every grid point if the grid is very sparse.

On top of these optimizations, there are several ways that defensive programming was implemented to ensure that the PopulationGrid object is used correctly. As with the rest of HAL, only the methods and functions that users are meant to interact with are exposed as public methods. The vast majority of the PopulationGrid internals, such as the arrays of populations and deltas, cannot be accessed directly. This ensures that the PopulationGrid object cannot be accidentally broken by the user. The PopulationGrid is also designed to fail immediately if a state is reached that doesn't make sense. If the population at any grid point would drop below 0, the model crashes and reports this so that the modeler can fix the issue. The binomial / multinomial solver will also crash the program if the probability / sum of the probabilities entered is greater than 1, or if a negative population size is entered. These failure states ensure that bugs are caught as close to when they first arise in the code as possible.

When we compare the performance (averaged from 20 repeats) of simple diffusion of a well mixed population on each grid type (using a random walk with probability 0.1 of moving to an adjacent position with a 3D Von-neumann stencil for the ABM and PBM, and forward Euler diffusion for the PDE), the benefit of PopulationGrids for large populations is clear (Figure 9). With one agent per grid point, the PopulationGrid model performs in similar time to the agent based model. However, as the number of agents per grid point increases by powers of 10, the ABM follows that increase in agents proportionally, while the PopulationGrid levels off at about 100 agents per grid point (when the "sequential coin flip" optimization is no longer likely to be used). The PDE, as expected, is unaffected by the change in agent count, as it uses the same floating point operations regardless.

3.4 PopulationGrid agent movement

Due to the novelty of the PBM method, many basic modeling assumptions had to be determined. One of these was how movement between lattice positions would work. In ABM tumor models,

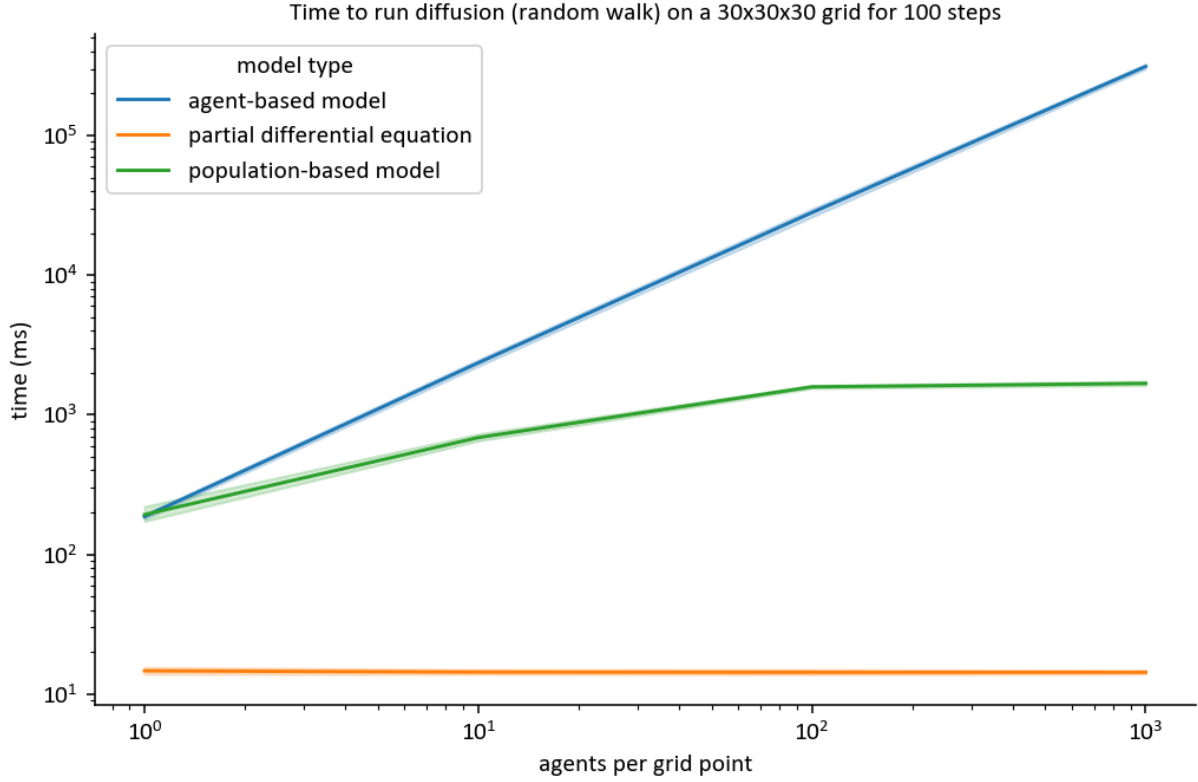


Figure 9. Performance comparison between PBM, ABM, and PDE grids in 3D. The ABM and PBM both used the 3D Von Neumann neighborhood to determine random walk positions, while the PDE used forward Euler. The number of agents per grid point was set as integer values in the ABM and PBM models, and was set as the local concentration at each grid point in the PDE model.

movement is not strictly necessary to grow a tumor, as new agents are placed adjacent to existing ones during cell proliferation. Movement can also be added to the ABM specifically to model cell migration. PDEs, however, require movement in order for the tumor to grow in space, as otherwise proliferation will only occur in locations that already contain cells. For this reason, diffusion/advection must serve the dual purpose of modeling cell migration and the spread of the tumor as a consequence of growth. For our PBM model, we decided on an analogous process to a PDE, with cells dividing into the same voxel that they started in, and a random movement "diffusion" of cells to model their spread in space. It is possible to tie this random walk to only the population of cells that divided on a given time-step to recapitulate some of the behavior of ABM growth without migration, but we decided on a PDE-like behavior for consistency with PDE models. PBM random movement is very similar to PDE diffusion at large populations, and

approaches it in the limit as the populations per voxel become larger, as was tested in the supplement of [18].

Solid tumors like those often found in non-small cell lung cancer are not very migratory compared to diffuse tumors as are often seen in glioblastoma. Since we intended to model non-small cell lung cancer in the subsequent project, we needed to find a migration function that would work well for modeling solid tumors. With fast growing and slow diffusing tumor models, the resulting tumor geometry is heavily dependent on the stencil used to model tumor cell spread. If a Von-Neumann stencil is used, the tumors tend to form in diamond shapes. If a Moore-neighborhood stencil is used, the tumors tend to form in box shapes. This can become a problem because the corners that jut out from the edge of the tumor will experience less competition for space and resources compared to the cells along the sides, since they are less surrounded by other tumor cells. This can lead to these corners being amplified over time, causing artifacts in the model behavior.

To attempt to find an optimal stencil for a spherical shape, we used a PDE as a proxy for PBM behavior for easier analysis. We made a PDE system that had a seed concentration of 1 in the center of a 3D lattice (29x29x29), and ran a growth and diffusion equation until the domain was 30% occupied with at least a concentration of 1E-6. The PDE was $\frac{dC}{dt} = gC(1 - C) + D\Delta C$ with g as a growth rate constant and D as a diffusion/random movement rate constant. The complexity comes into how diffusion is applied, the Δ stencil, which we modified to look for an optimal spherical shape.

We first examined different options available for a diffusion stencil. For performance and to ensure locality of movement, we limited our exploration to the Moore neighborhood in 3D, a 3x3x3 cube of the 26 closest voxels to one central voxel. In this example, all cells have proliferation rate $g = 1$ and movement rate $D = 1e^{-6}$. The Moore neighborhood stencil can be broken up into 3 sets where all voxels contained in each set are equidistant from the center voxel (Figure 10A). The first set is the voxels in the 6 cardinal directions, known as the Von-Neumann neighborhood (green). The second set is the 12 voxels that are each adjacent to the Von-Neumann neighborhood and make up part of the Moore neighborhood (blue). The third set is all of the remaining corner voxels, of which there are 8, which complete the Moore neighborhood (red). Let D be a normalized diffusion rate constant. We will encode a diffusion rate for each of these sets as

a vector with 3 components (diffusion rate of the 6 voxel set, diffusion rate of the 12 voxel set, diffusion rate of the 8 voxel set). The Von-Neumann neighborhood is constructed by setting the diffusion rate of 6 voxel set to D , and the other diffusion rates to 0: $(D, 0, 0)$. The Moore neighborhood is (D, D, D) . The distance-based neighborhood used in the subsequent chapter is $(D, \frac{D}{\sqrt{2}}, 0)$. In the mesoscopic model [27], $(D, \frac{D}{\sqrt{2}}, \frac{D}{\sqrt{3}})$ was used. In that case the modelers were representing glioblastoma, which led to them having a faster movement rate compared to proliferation rate, which is why their system was less sensitive to these considerations. We zeroed the last component in our version because a nonzero value would make the tumors more square. Finally the empirical "most circular" stencil in this case is $(D, 0.00631D, 0.00008D)$. Figure 10B shows a 2D cross section of the shapes that these stencils will form.

The last set of values were found empirically by a computational sweep of possible diffusion rates for the 12 and 8 voxel neighborhoods, and choosing the "best" one based on a circularity metric. Here circularity is defined as $\frac{r}{\sqrt[3]{v}}$ where r is the distance to the furthest edge of an object from its center of mass, and v is the object's volume. The "object" is defined as the region of the PDE where the concentration is above $1e-6$. This metric is a scale invariant way to determine how spherical an object is, because of the cubic relationship between the volume and the radius of a sphere. Also, since the shape defined by the PDE grows over time and the circularity shifts, we consider the average circularity over the full model duration to score a given diffusion stencil.

In order to ensure that the tests we conducted would generalize well, we first tested how circularity would change given different movement rates and proliferation rates (Figure 10C). We see that throughout the domain the circularity metric is the same along the diagonal in which the ratio of movement rate and proliferation rate are constant. Given that the behavior of the circularity metric depends on this ratio and not particular values of movement and proliferation, we proceed to finding optimal values for different ratios of movement and proliferation.

The ratio values were constructed by pairing all permutations of proliferation rates in the range $10^{-x/10}, x = 0, 1, \dots, 9$ and movement rates in the range $10^{-x}, x = 3, \dots, 7$. The log of the ratio between all permutations of those ranges steps evenly along 2.1, 2.2, ..., 6.9, 7.0. Across these ratios, we saw that the best parameter values followed a consistent trajectory, which we fit linearly, leading to an equation for optimally spherical tumors (Figure 10D-F). The parameter values that we swept (the y axis in the plots) were the ratios of the diffusion rate in the 6 voxel set to the 12

voxel set, and the 12 voxel set to the 8 voxel set. We chose ranges for these ratios from trial and error to be $10E-1.0$, $10E-1.1$, ... $10E-2.9$, $10E-3$. Searching for appropriate ratios of these diffusion rates for the different sets allowed us to only consider stencils with D values for the 12 voxel set that were lower than the D value for the 6 voxel set, and only D values for the 8 voxel set that were lower than the D value for the 12 voxel set, which makes sense given the relative distances from the central voxel to the voxels in these sets. This analysis showed clear trends that we fit linearly with trendlines that map the movement/proliferation rate ratio to the diffusion rate ratio between voxel sets that will produce a spherical tumor.

We lastly observe the relative performances of these schemes with different proliferation and movement rate ratios, and compare them to the earlier examples (Figure 10G). We see that the stencil with 26 points (the full moore neighborhood) consistently outperforms the 18 point stencil. The horizontal lines show the performance of Von-Neumann, Moore, distance-based, and empirical methods at a proliferation to movement rate ratio of $1E6$.

3.5 Generic cycle for population-based modeling

The following two chapters use the PBM approach for their cellular components. A generic underlying structure was used as a skeleton for the projects which we will outline here (Figure 11). We prioritized modularity in designing the structure, meaning that individual assumptions that govern probabilities of events like proliferation and death can be added or removed without affecting the underlying structure, thus shortening time spent modifying the model to prototype different assumptions. We also prioritized efficiency, ensuring that multiple rules were compressed together into single binomial/multinomial computations whenever possible. Each cycle simulates one day, as a day is a good rule-of-thumb for the cell cycle time of human cells.

At the beginning of the day cycle, several events occur in sequence: cell death, proliferation, and mutation. All of these events take place within the individual voxels. Because of this, we can evaluate these steps on a per-voxel basis, track how cell numbers are fluctuating, then update the PBM populations with the results of all these events once they are all completed. Consider an example population of cells at time t and voxel i : $n_{i,t}$, and a set of d death probability calculating functions $\bar{f}^d$ that each calculate the probability of death from a given factor, such as random death, starvation, drug exposure, etc. We compute the number of cells that will die, $P(\text{death}_{i,t})$

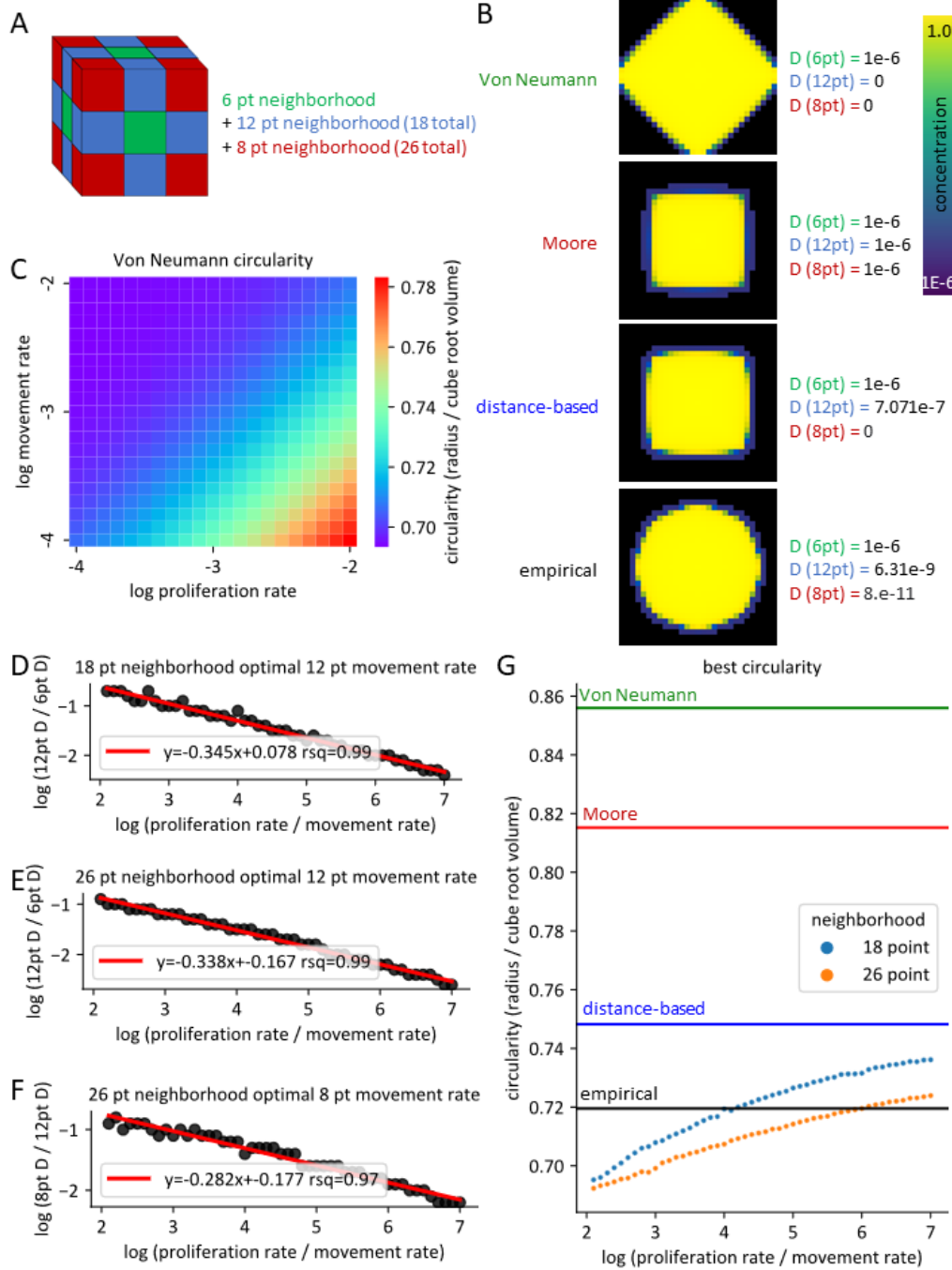


Figure 10. **A** Cube graphic shows different sets of voxels that together make up the 3D Moore neighborhood. **B** PDE visualizations using different diffusion stencils with proliferation rate 1.0. Black indicates concentration less than $1E-6$. **C** Plot of circularity metric for different movement and proliferation rates **D** Scatter plot shows best fitting ratio of 12 pt diffusion rate to 6 pt diffusion rate at different proliferation to movement rate ratios. **E-F** Scatter plots show best fitting ratios for 12 pt diffusion rate to 6 pt, and 8pt diffusion rate to 12pt. **G** Scatter points show best fit circularity metric results for different proliferation rate to movement rate ratios. Colored horizontal lines show circularity metric scores from stencils in B with a proliferation to movement rate ratio of $1E6$.

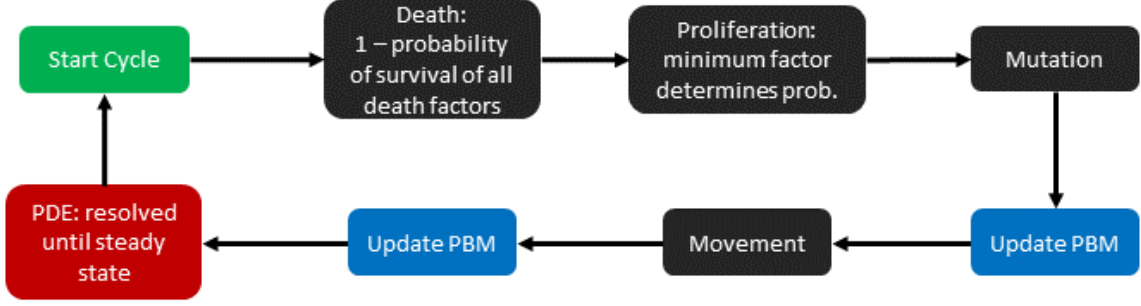


Figure 11. The generic PBM loop that is used in the next two chapters. Each cycle of the loop simulates one day. The green box shows where the PBM cycle starts. The black boxes show computations that involve adding changes to the PopulationGrids. The blue boxes show when the PopulationGrids are updated with the previous changes so that future calculations will take them into account. The Red box shows where the PDE components are updated.

by multiplying the probabilities of survival (1 - probability of death) of all death probability calculating functions to get the probability of survival of all of them. Each factor considers the state of the model at position i and time t which we will denote as $m_{i,t}$. We then take 1 - probability of survival of all factors to get the overall probability of death:

$$P(\text{death}_{i,t}) = 1 - \left(\prod_{j=1}^d 1 - f_j^d(m_{i,t}) \right)$$

This probability is then plugged into the binomial to get the number of death events that will occur: $d_{i,t}$, such that we can find the number of survivors as $n_{i,t} - d_{i,t}$. From this population of survivors, we get a number that will divide $v_{i,t}$ by finding the minimum probability from a set of all p proliferation probability factor functions $\bar{f}^p$. The minimum probability factor is considered the rate limiting factor, and thus the proliferation probability takes on its value:

$$P(\text{proliferation}_{i,t}) = \min_{j=1}^p [f_j^p(m_{i,t})]$$

This probability is then used with the binomial function to get a number of dividing cells $v_{i,t}$.

After all proliferation and death events are accounted for, we next look at mutation, which is similar to movement but cells are moved between PopulationGrids instead of between voxels. In our model we only allow cells that proliferated to mutate, so the population that can mutate on a

given timestep is $2v_{i,t}$, although this assumption could be changed to simply include all cells that exist in the voxel to capture changes to cell phenotype that are not proliferation dependent. Once again, the form that governs the probabilities of mutation can depend on many factors, but the sum of the probabilities of mutation cannot exceed 1. In the next chapter, there is a fixed probability of mutation between glycolytic and angiogenic phenotypes. From the multinomial we get a number of cells that will mutate, meaning the number that will be transferred to different PopulationGrids $m_{i,t}$. These transitions along with all birth and death events are applied with an update at the end of mutation, such that at the end of this sequence, the number of cells of that population in the voxel will be: $p_{i,t} - d_{i,t} + v_{i,t} - m_{i,t}$, plus any cells at voxel i that mutated into this phenotype from other PopulationGrids in the model.

The cycle continues with cell movement. In general, movement from a position will depend on some set of probabilities that are each associated with the voxels in a local neighborhood. This set of probabilities cannot sum to over 1, but otherwise can be flexibly determined by factors such as the concentrations of chemo-attractants or the existence of boundaries that inhibit movement. These factors can be combined to alter the probabilities of migration into different voxels of the neighborhood. The multinomial is then sampled to get the number of agents that will move to each new location. In the model we used in the subsequent chapters, we only considered local density in determining movement probabilities, so we did not have to develop a way of combining multiple movement influencing factors. We will elaborate on the density-based migration equation further in the next chapter. After the number of agents that will move is calculated over the entire grid, the PBM is updated so that the populations reflect the placement of the agents in their new contexts.

Putting death before proliferation causes death to supersede proliferation, meaning that cells cannot divide if they will die at a given location. Thus it's impossible for generations of cells to persist in environments that should kill them. In this framework we assume migration happens after death, proliferation, and mutation, which allows the density-dependent movement to respond to changes in voxel density that occur on the same timestep. In future PBM models, it may be useful to move the migration step to occur before death, proliferation, and mutation. This would ensure that the cells will respond to their new environments during the same timestep that they

enter them, so if moving to a new voxel should always result in death or mutation for example, it will occur immediately.

After the PBM is updated with the results of movement, the PDEs that are used along with the PBM are updated by iterating the PDEs until the maximum change after each cycle falls below some error bound ϵ at which point we consider the PDEs to be at steady-state. This is useful because the timescales of cells and diffusibles are very different, so the only way to integrate them together is by resolving the PDEs to steady state after each day-step and then advancing to the changes in the PBMs for the following day.

As explained earlier, this framework is the algorithmic substrate on which the models in the next two chapters are built, but this framework is flexible by design such that a variety of models could be implemented using it.